

Navigation and record-page kit: rules, prompts, commands, code, AI skills, agents and hooks for any application

Every record shown anywhere is a way into that record, and every panel finishes loading. This kit prevents and fixes unclickable records, dead-end tabs, loaders that never finish, empty overview tabs and clipped detail pages, in any sector and on any stack.

Companion to the Application Build Playbook (Section 7.13). Kit folder: No-Dead-Ends-Kit (SKILL.md, AGENTS.md, subagents, slash commands, hooks, templates, audit script).

Contents

Contents	2
1. How to use this kit	2
2. The problem pattern.....	3
3. Rules (paste into any specification)	4
4. Master prompt.....	4
5. Step prompts (run one at a time)	5
6. Audit commands.....	6
6.1. Run the dead-end audit script	6
6.2. Find hand-written record URLs	7
6.3. Find loaders that may hang	7
6.4. Check every entity has a detail route and a by-id endpoint	7
6.5. Smoke-test a record endpoint as each role	7
6.6. Run the guards.....	7
7. Reference implementation (adapt to your stack).....	7
7.1 Route registry: the only place record URLs are defined	7
7.2 RecordLink: permission-aware, real anchor	8
7.3 Clickable table row.....	8
7.4 API references: always {type, id, label}.....	9
7.5 Panels that always resolve (timeout, empty, error)	9
7.6 Detail page layout	10
7.7 E2E navigation test (Playwright)	10
8. The audit script	11
9. AI tooling files (.md): skill, agents, commands, rules.....	13
9.1 Install into a project	13
9.2 SKILL.md	14
9.3 AGENTS.md	15
10. Hooks: automatic enforcement	16
10.1 .claude/settings.json.....	16
10.2 PostToolUse hook: .claude/hooks/nav-check-edit.mjs	17
10.3 Stop hook: .claude/hooks/nav-check-stop.mjs	17
10.4 Git pre-commit hook and the baseline	18
11. Navigation map template	19
12. Definition of Done: navigation.....	19
13. Test scenarios	19

1. How to use this kit

This kit fixes and prevents one of the most common defects in business applications: **records that are shown but cannot be opened**. It also covers the detail-page faults that usually travel with it: loaders that never finish, empty

overview tabs and clipped content. It is stack-neutral. The reference code uses React, React Router and Playwright, and translates directly to Vue, Angular, Next.js, Django, Rails, Spring or .NET.

The kit comes in two forms. This document is for people; the **kit folder** is for AI coding tools and CI. The folder holds a Claude Code skill, subagents, slash commands, hooks, an AGENTS.md for other AI tools, a Cursor rule, a git hook, code templates and an audit script. Both forms are generated from the same source.

Situation	What to do
New application	Install the kit folder (Section 9). Add the rules (Section 3) to the specification. Build the route registry and RecordLink (Section 7) in the first slice, before any list screen. The hooks then stop dead ends from ever appearing.
Existing application	Install the kit and record today's findings as the baseline (Section 10.4). Run /nav-audit, then /nav-fix. Alternatively, paste the Master Prompt (Section 4, mode AUDIT+FIX) or run the step prompts (Section 5).
Any AI tool other than Claude Code	Copy AGENTS.md (Codex, Cursor, Copilot, Gemini and others) and paste the Master Prompt.
Before every release	Run the audit script and the navigation E2E test (Section 6), and tick the Definition of Done (Section 12).

The one-line rule

No dead ends: every record shown anywhere is a way into that record, and every panel finishes loading.

2. The problem pattern

A typical case: a customer's 360 page shows an Assets table (registration, model, expiry, value). The registration numbers look like identifiers but are plain text, and clicking the row does nothing. Next to it, the Timeline panel shows grey skeleton bars forever. The Overview tab contains only that one table, tabs with zero records look the same as full ones, and a header badge is cut off at the right edge. The same pattern repeats on many screens.

#	Symptom	Root cause	Rules
S1	Identifiers and rows in tables cannot be clicked	Each screen renders fields as text; no shared way to link an entity	NAV-01, NAV-02, NAV-04
S2	Related-record tabs on a 360 page are dead ends	Child tables built without links; child detail pages missing	NAV-03, NAV-06, NAV-12
S3	Loading skeleton never resolves	Request failed, returned empty, endpoint missing, or no timeout/error branch	NAV-10
S4	Overview tab shows only one table	No definition of what an overview must answer	NAV-14
S5	Zero-count tabs look like populated ones	Tabs not ordered or styled by content	NAV-15
S6	Content clipped at the right edge	Fixed widths / no overflow container with the sidebar open	NAV-17
S7	Users copy IDs into global search to reach a record	Consequence of S1 and S2	NAV-01 to NAV-08

Measured on a real codebase

When the audit script from this kit was run on a production CRM client that had this exact problem, it reported **47 dead ends and 3 hand-written record URLs** across list pages, 360 tabs, dashboards, HR screens and global search results. The two asset tables from the screenshot were among them. Recording those findings as a baseline made the check pass immediately, so from that point only new dead ends fail.

3. Rules (paste into any specification)

ID	Level	Rule
NAV-01	MUST	Every entity with a detail page is a link wherever it is shown: table cells, rows, cards, 360 tabs, timeline/activity items, notifications, search results, audit rows, approval requests, alerts.
NAV-02	MUST	One route registry maps entity type to URL + required permission. All links are built from it via one component/helper (RecordLink / linkTo). No hand-written record URLs in screens.
NAV-03	MUST	360 pages link both ways: parent tabs open each child; each child's header links back to its parent(s). Tab counts equal the rows shown.
NAV-04	MUST	The whole row is clickable AND the primary identifier is a real anchor (keyboard, screen reader, middle-click, open-in-new-tab, copy link). Row actions (buttons, checkboxes, menus) stop propagation.
NAV-05	MUST	Links respect permissions: no access means plain text (optional lock icon + tooltip), never a link to a 403/404. The server still enforces access at the destination.
NAV-06	MUST	Every entity type has a routable detail URL that works on direct load, refresh, bookmark and share.
NAV-07	MUST	APIs return references as { type, id, label }, and notification/timeline/audit rows store referenceType + referenceId, so the client can always link.
NAV-08	MUST	Back returns to the list with the same filters, sort, page and scroll position (list state in the URL).
NAV-09	MUST	Breadcrumbs or a clear back path on every detail page (List › Parent › Record).
NAV-10	MUST	Every panel, tab and widget resolves to data, empty, error-with-retry or forbidden within a timeout (10–15 s). An endless skeleton is a defect.
NAV-11	MUST	KPI tiles and chart segments drill down to the filtered list, whose rows then open records.
NAV-12	MUST	An entity without a detail page yet shows plain text and is logged in the gap register, never a link to a blank page.
NAV-13	MUST	Standard detail header: icon/avatar, name, business ID (copy button), status, owner, 3–5 key facts, primary actions on the right, secondary actions in "More".
NAV-14	MUST	The Overview answers "who is this, what is open, what happens next": key details, open items (each linked), next action, latest activity.
NAV-15	MUST	Tabs ordered by use, each with a count; zero-count tabs muted but reachable, opening an empty state with the create action.
NAV-16	MUST	The timeline merges all related events, newest first, filterable, paginated, each item linked, with its own loading/empty/error states.
NAV-17	MUST	Nothing clipped at 1366 px with the sidebar open; wide content scrolls inside its own container.
NAV-18	SHOULD	Hover quick-view for linked records in dense tables; "recently viewed" list; copy-ID affordance.
NAV-19	SHOULD	CI audit fails when an entity identifier is rendered outside RecordLink; an E2E test opens the first row of every list and every 360 tab for every role.
NAV-20	MUST	Merged or renamed records keep old IDs as aliases so existing links and bookmarks still resolve.

4. Master prompt

Paste into your AI coding session inside the project. Set MODE to NEW for a new build, or AUDIT+FIX for an existing application. In Claude Code with the kit installed, /nav-audit followed by /nav-fix does the same.

```
MODE: {{NEW | AUDIT+FIX}}
Project: {{name}} - stack: {{e.g. React + Express + MongoDB}}
Roles: {{admin, manager, agent, ...}}

GOAL: No dead ends. Every record shown anywhere must open its own record,
every panel must finish loading, and every detail page must follow a standard
layout. Rules NAV-01..NAV-20 of the "No Dead Ends" kit are binding.

DO, IN ORDER - stop after steps 1-3 for my approval:
```

1. INVENTORY: list every entity type (customer, asset, order, invoice, ticket, employee, document, ...). For each: detail route? permission? API that returns it by id? Output a table. Then list every place each entity is displayed (tables, 360 tabs, cards, timeline items, notifications, search results, audit rows) with file:line and whether it is linked. Give counts: displayed / linked / dead / hand-written URL. Start with `node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src`, then read the screens.
2. LOADERS: find every loading state; mark those that can hang (no timeout, no error branch, no empty branch, missing or failing endpoint).
3. PLAN: route registry, RecordLink component, list state in the URL, detail-page header/overview/tabs/timeline layout. List missing detail pages and missing endpoints separately.
4. FIX: create registry + RecordLink (permission-aware, real anchor, data-record-link attribute); route EVERY occurrence through it; clickable rows; breadcrumbs; list state in URL; every hanging loader gets timeout + empty + error-with-retry; build missing detail pages or log them as gaps (plain text); APIs return {type,id,label} references.
5. DETAIL PAGES: standard header, Overview (who / what is open / what next), tabs by frequency with counts and muted zero tabs, merged linked timeline, nothing clipped at 1366 px with the sidebar open.
6. GUARD: CI audit script + Playwright test that opens the first row of every list and every 360 tab as every role; no aria-busy after 15 s.
7. VERIFY + REPORT: run build, tests and the audit; show real output. Report before/after counts, remaining gaps, and anything not verified.

RULES: the server still enforces access on every detail endpoint (out-of-scope record -> 404). No hand-written record URLs. Never claim "fixed" without test output. Keep changes minimal and in the existing code style.

5. Step prompts (run one at a time)

P1. Entity and route inventory

List every entity type in this codebase. For each give: detail route (or MISSING), permission required, API endpoint returning one record by id (or MISSING), and the field used as its human label/business ID. Output a table. Read the client router and the API routers; cite file:line. No code changes.

P2. Dead-end audit

Run: `node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src`
Then find every place a record of any entity type is displayed (the script is a heuristic - also read list, detail, dashboard, search and notification screens): table cells, row renderers, cards, 360 tabs, timeline items, notifications, search results, audit rows. For each: file:line, entity type, linked (yes / no / hand-written URL). End with counts per screen and total. No code changes.

P3. Hanging-loader audit

Find every loading state (skeleton, spinner, isLoading, aria-busy). For each, check: error branch? empty branch? timeout/abort? does the endpoint exist and return 2xx for every role? List the ones that can hang and why (file:line). Reproduce suspected hangs by calling the endpoint. No code changes.

P4. Build the link foundation

Create the route registry (entity type -> path builder + permission) and a

permission-aware RecordLink component that renders a real anchor with data-record-link, stops click propagation, and renders plain text when the user lacks permission or the id is missing. An unknown entity type must throw. Add unit tests and show them pass.

P5. Replace every dead end

Using the P2 list, route every occurrence through RecordLink (including the hand-written URLs). Make list rows clickable via linkTo while keeping the anchor on the identifier. Add breadcrumbs to detail pages. Keep list filters/sort/page in the URL. Re-run the audit script; show before/after.

P6. Fix loaders and APIs

Fix every loader from P3: 12 s timeout, empty state with create action, error state with retry and request ID. Where the cause is server-side (missing endpoint, 500, wrong scope) fix the endpoint. Make APIs return related records as {type, id, label}. Show each panel resolving.

P7. Standardise detail pages

Apply the standard layout to every entity detail page: header (name, business ID with copy, status, owner, key facts, primary actions), Overview (key details, open items linked, next action, latest activity), tabs by use with counts and muted zero tabs, merged linked timeline, nothing clipped at 1366 px with the sidebar open. Screenshot before/after.

P8. Guard and verify

Add the kit's check-record-links.mjs to CI (with --baseline for an existing app), enable the hooks, and add the Playwright navigation test (every list's first row + every 360 tab, every role, no aria-busy after 15 s). Run build, unit, API, E2E and the audit. Report real output, before/after counts, and remaining gaps (logged in the gap register).

6. Audit commands

Run from the repository root. Adjust client/src / server/src and field names to the project. rg (ripgrep) runs on Windows, macOS and Linux; PowerShell equivalents are given. The commands find candidates quickly; the step prompts do the exhaustive review.

6.1. Run the dead-end audit script

```
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src
# DEAD END = identifier rendered with no link (NAV-01)
# HAND URL = linked with a hand-written URL instead of the registry (NAV-02)
# Exit code 1 when anything is found - use it as a CI gate.

# Existing app: record today's findings once, then fail only on NEW ones
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --write-baseline .nav-baseline.json
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --baseline .nav-baseline.json
```

6.2. Find hand-written record URLs

```
rg -n --glob "*.js,jsx,ts,tsx"
"(to|href)=\{?\s*[\`'\"]/(customers|assets|vehicles|orders|invoices|policies|tickets|employees)/"
client/src
```

```
Get-ChildItem client/src -Recurse -Include *.js,*.jsx,*.ts,*.tsx |
  Select-String -Pattern
"(to|href)=\{?\s*[\`'\"]/(customers|assets|vehicles|orders|invoices|policies|tickets)/"
```

6.3. Find loaders that may hang

Files that show a loading state but never handle an error state:

```
rg -l --glob "*.jsx,tsx" "isLoading|loading|Skeleton|aria-busy" client/src \
| xargs rg -L "isError|error\b|onError|catch\("
```

```
Get-ChildItem client/src -Recurse -Include *.jsx,*.tsx | Where-Object {
  (Select-String -Path $_ -Pattern 'isLoading|loading|Skeleton|aria-busy' -Quiet) -and
  -not (Select-String -Path $_ -Pattern 'isError|error\b|onError|catch\(' -Quiet)
} | Select-Object FullName
```

6.4. Check every entity has a detail route and a by-id endpoint

```
# Client routes with an :id parameter (React Router / Vue Router)
rg -n "path[=:]s*[\`'\"][^\\`'\"]*:w*[iI]d" client/src

# Server one-record endpoints (Express / Fastify)
rg -n "\.get\(\s*[\`'\"][^\\`'\"]*/:id[\`'\"]" server/src
```

6.5. Smoke-test a record endpoint as each role

```
for role in admin manager agent; do
  curl -s -o /dev/null -w "$role %{http_code}\n" -b cookies-$role.txt \
    "http://localhost:4000/api/{{entity}}/{{id}}"
done
# Expected: 200 for roles in scope, 404 (not 403, not 500) for roles out of scope.
```

6.6. Run the guards

```
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --baseline .nav-
baseline.json
npx playwright test e2e/navigation.spec.js
npx playwright test e2e/navigation.spec.js --headed      # watch it click through
git config core.hooksPath .githooks                      # enable the pre-commit hook (once per
clone)
```

7. Reference implementation (adapt to your stack)

7.1 Route registry: the only place record URLs are defined

File: `.claude/skills/no-dead-ends/templates/entityRoutes.js`

```
// client/src/lib/entityRoutes.js
// One map from entity type to URL and permission. Adding an entity or moving
// a route is a one-line change here, and every table, card and timeline
// follows automatically.
export const ENTITY_ROUTES = {
  customer: { path: (id) => `/customers/${id}`, permission: 'customers.view' },
  asset: { path: (id) => `/assets/${id}`, permission: 'assets.view' },
  order: { path: (id) => `/orders/${id}`, permission: 'orders.view' },
  invoice: { path: (id) => `/invoices/${id}`, permission: 'invoices.view' },
  ticket: { path: (id) => `/tickets/${id}`, permission: 'tickets.view' },
  employee: { path: (id) => `/employees/${id}`, permission: 'employees.view' },
};

export function linkTo(type, id) {
  const route = ENTITY_ROUTES[type];
  // Throw, don't return '#': an unknown type is a developer error, and a
  // silent '#' link is exactly the dead end this registry exists to prevent.
  if (!route) throw new Error(`linkTo: no route registered for entity type "${type}"`);
  return route.path(encodeURIComponent(id));
}
```

7.2 RecordLink: permission-aware, real anchor

File: `./claude/skills/no-dead-ends/templates/RecordLink.jsx`

```
// client/src/components/RecordLink.jsx
import { Link } from 'react-router-dom';
import { ENTITY_ROUTES, linkTo } from '../lib/entityRoutes';
import { useSession } from '../lib/auth';

export function RecordLink({ type, id, children, className = '' }) {
  const { can } = useSession();
  const route = ENTITY_ROUTES[type];
  if (!route) throw new Error(`RecordLink: unknown entity type "${type}"`);

  // No id or no access: plain text, never a link that ends on 403/404.
  if (!id || !can(route.permission)) {
    return <span className={className}>{children}</span>;
  }
  return (
    <Link
      to={linkTo(type, id)}
      data-record-link={type}
      className={`text-brand-700 hover:underline ${className}`}
      // The row is clickable too; stop the event so one click = one navigation.
      onClick={(e) => e.stopPropagation()}
    >
      {children}
    </Link>
  );
}
```

7.3 Clickable table row

File: `./claude/skills/no-dead-ends/templates/ClickableRow.jsx`

```
// Example: a list row that opens its record from anywhere on the row,
// while the identifier stays a real anchor.
import { useNavigate } from 'react-router-dom';
import { RecordLink } from '../components/RecordLink';
import { linkTo } from '../lib/entityRoutes';

export function AssetRow({ asset }) {
  const navigate = useNavigate();
```



```

return (
  <tr
    className="cursor-pointer hover:bg-gray-50"
    onClick={() => navigate(linkTo('asset', asset.id))}
  >
    { /* The anchor gives keyboard, screen-reader and open-in-new-tab support. */ }
    <td><RecordLink type="asset" id={asset.id}>{asset.registration}</RecordLink></td>
    <td>{asset.model}</td>
    <td>{formatDate(asset.expiryDate)}</td>
    <td className="text-right tabular-nums">{formatMoney(asset.value)}</td>
    <td onClick={(e) => e.stopPropagation()}>{ /* row action menu */}</td>
  </tr>
);
}

```

7.4 API references: always {type, id, label}

File: `.claude/skills/no-dead-ends/templates/api-reference-shape.json`

```

{
  "ok": true,
  "data": {
    "customer": { "id": "c-060", "number": "CUS-000060", "name": "Example Transports" },
    "assets": [
      {
        "ref": { "type": "asset", "id": "a-118", "label": "MH 02 HH 1118" },
        "model": "Tata Ace Gold",
        "expiryDate": "2026-09-29",
        "value": 385000
      }
    ],
    "timeline": [
      {
        "at": "2026-09-20T10:15:00Z",
        "kind": "CALL",
        "ref": { "type": "call", "id": "call-771", "label": "Outbound call, 4m 12s" }
      }
    ]
  }
}

```

7.5 Panels that always resolve (timeout, empty, error)

File: `.claude/skills/no-dead-ends/templates/usePanel.js`

```

// client/src/lib/usePanel.js
// Every panel ends in ready, empty or error - never an endless skeleton.
import { useEffect, useState } from 'react';

const isEmpty = (d) => d == null || (Array.isArray(d) && d.length === 0);

export function usePanel(fetcher, deps, { timeoutMs = 12000 } = {}) {
  const [state, setState] = useState({ status: 'loading' });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort('timeout'), timeoutMs);
    setState({ status: 'loading' });
    fetcher(ctrl.signal)
      .then((data) => setState(isEmpty(data) ? { status: 'empty' } : { status: 'ready', data }))
      .catch((err) => {
        if (ctrl.signal.reason === 'unmount') return;
        setAttempt({

```

```

        status: 'error',
        message: ctrl.signal.reason === 'timeout' ? 'This is taking too long.' : err.message,
        requestId: err.requestId,
    });
    });
    .finally(() => clearTimeout(timer));
    return () => { clearTimeout(timer); ctrl.abort('unmount'); };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [...deps, attempt]);

  return { ...state, retry: () => setAttempt((n) => n + 1) };
}

// Usage: every branch renders something final.
//
// function TimelinePanel({ customerId }) {
//   const t = usePanel((signal) => api.get(`/customers/${customerId}/timeline`, { signal })),
//   [customerId];
//   if (t.status === 'loading') return <Skeleton rows={5} aria-busy="true" />;
//   if (t.status === 'error')   return <PanelError message={t.message} requestId={t.requestId}
//   onRetry={t.retry} />;
//   if (t.status === 'empty')   return <EmptyState text="No activity yet." />;
//   return <Timeline items={t.data} />; // each item renders <RecordLink {...item.ref} />
// }

```

7.6 Detail page layout

File: `.claude/skills/no-dead-ends/templates/DetailPage.jsx`

```

// Standard detail (360) page layout - adapt component names to your UI kit.
<DetailPage breadcrumbs={[{ label: 'Customers', to: '/customers' }, { label: c.name }]}>
  <DetailHeader
    avatar={initials(c.name)} title={c.name} businessId={c.number} // with copy button
    status={c.status}
    owner={<RecordLink type="employee" id={c.ownerId}>{c.ownerName}</RecordLink>}
    facts={[c.phone, c.email, c.city]}
    primaryActions={[newOrder, call]} moreActions={[edit, merge, deactivate]}
  />
  <Tabs urlParam="tab" items={orderByUsage(tabsWithCounts)} muteZero />
  {tab === 'overview' && (
    <Overview>
      <KeyDetails record={c} />
      <OpenItems items={openItems} /> // /* each item is a RecordLink */
      <NextAction action={c.nextAction} />
      <TimelinePanel customerId={c.id} limit={10} />
    </Overview>
  )}
</DetailPage>
// Layout: the content column is min-w-0 and wide tables sit in overflow-x-auto,
// so nothing is clipped at 1366 px with the sidebar open.

```

7.7 E2E navigation test (Playwright)

File: `.claude/skills/no-dead-ends/templates/navigation.spec.js`

```

// e2e/navigation.spec.js - every list's first row and every 360 tab opens a record, for every
// role.
import { test, expect } from '@playwright/test';
import { loginAs } from './helpers';

const ROLES = ['admin', 'manager', 'agent'];
const LISTS = ['/customers', '/assets', '/orders', '/invoices', '/tickets'];

const escapeRegExp = (s) => s.replace(/[\.\*\?\^\$\{\}\(\)\[\]\\\]/g, '\\$&');

```

```

for (const role of ROLES) {
  for (const list of LISTS) {
    test(`${role}: first row of ${list} opens its record`, async ({ page }) => {
      await loginAs(page, role);
      await page.goto(list);
      const rows = page.locator('table tbody tr');
      // An empty list proves nothing - fail loudly instead of passing green.
      await expect(rows.first(), `${list} has no rows for ${role}`).toBeVisible();
      const link = rows.first().locator('a[data-record-link]').first();
      await expect(link, 'row has no record link (dead end)').toBeVisible();
      await link.click();
      await expect(page.getByTestId('detail-header')).toBeVisible();
      await expect(page.locator('[aria-busy="true"]')).toHaveCount(0, { timeout: 15000 });

      // Every 360 tab: open it, wait for it to resolve, open its first row if any.
      const tabs = page.getByRole('tab');
      const tabCount = await tabs.count();
      for (let i = 0; i < tabCount; i++) {
        await tabs.nth(i).click();
        await expect(page.locator('[aria-busy="true"]')).toHaveCount(0, { timeout: 15000 });
        const childLink = page.locator('[role="tabpanel"] a[data-record-link]').first();
        if (await childLink.count()) {
          const href = await childLink.getAttribute('href');
          await childLink.click();
          await expect(page).toHaveURL(new RegExp(escapeRegExp(href)));
          await page.goBack();
        }
      }
    });
  }
}

```

8. The audit script

check-record-links.mjs finds identifiers rendered without the registry. It has been tested on a real codebase, and it handles both inline rendering and column-definition tables. Findings are reported as **DEAD END** (no link at all) or **HAND URL** (linked, but with a hand-written URL). The script exits with code 1 when anything is found. With --baseline, it fails only on findings that are new since the baseline was recorded. Tune ENTITIES to your domain.

File: .claude/skills/no-dead-ends/scripts/check-record-links.mjs

```

// check-record-links.mjs
// Heuristic guard against dead ends: flags JSX/TSX/Vue/Svelte lines that
// render an entity's identifier without going through RecordLink / linkTo.
//
// Catches the two common shapes:
//   inline: <td>{row.invoiceNumber}</td>
//   columns: { key: 'registrationNumber', render: (r) => r.registrationNumber }
//
// Reports two kinds of finding:
//   DEAD END - identifier rendered with no link at all (rule NAV-01)
//   HAND URL - linked, but with a hand-written URL instead of the registry (NAV-02)
//
// Accept a deliberate exception (e.g. a record's own ID in its own header)
// with a trailing comment on the line: // nav-ok: <reason>
//
// Usage:
//   node check-record-links.mjs [paths...]                files or folders (default: client/src)
//   node check-record-links.mjs client/src --write-baseline .nav-baseline.json
//   node check-record-links.mjs client/src --baseline .nav-baseline.json
//
// With --baseline, only findings NOT in the baseline fail. That lets an existing
// app adopt the guard today (no new dead ends) and burn the old ones down.

```

```
// Baseline keys are file + line text, so they survive lines moving up or down.
// Exit code: 0 = pass, 1 = findings.
import { readFileSync, writeFileSync, readdirSync, statSync, existsSync } from 'node:fs';
import { join, relative } from 'node:path';

// Tune to your domain: entity prefixes whose identifiers must be links.
const ENTITIES = 'customer|client|patient|student|member|lead|quote|order|invoice|receipt|payment|'
  + 'policy|claim|case|ticket|booking|asset|vehicle|property|unit|product|'
  + 'employee|vendor|supplier|contract|project|task|document';
const SUFFIX = '(:Number|No|Code|Ref)';
const INLINE = new RegExp(`\\.(?:${ENTITIES})${SUFFIX}|registration(?:Number)?\\b`, 'i');
const COLUMN = new
RegExp(`\\b(?:key|accessor|field|dataIndex)\\s*:\\s*["`](?:${ENTITIES})${SUFFIX}|registration(?:Number)?["`]'i`);
const REGISTRY = /RecordLink|linkTo\\(|nav-ok:/;
const HAND_LINK = /<Link\\b|<a\\s[^]*href=|\\bhref=|navigate\\(|router\\.push\\(/;
const IGNORE = /console\\.|\\s*(\\|\\/|\\*|\\/\\*)|\\s*import\\s|value=\\{|placeholder=/;
const UI_FILE = /\\.jsx|tsx|vue|svelte$/;
const LOOKAHEAD = 2; // column definitions often put the render/link on the next lines

const args = process.argv.slice(2);
const flag = (name) => { const i = args.indexOf(name); return i >= 0 ? args.splice(i, 2)[1] : null; };
const writeBaseline = flag('--write-baseline');
const baselineFile = flag('--baseline');
const roots = args.length ? args : ['client/src'];

const files = (p) => {
  if (!existsSync(p)) return [];
  if (statSync(p).isFile()) return UI_FILE.test(p) ? [p] : [];
  return readdirSync(p).flatMap((f) => (f === 'node_modules' || f.startsWith('.') ? [] : files(join(p, f))));
};

export function scanFile(file) {
  const findings = [];
  const lines = readFileSync(file, 'utf8').split('\\n');
  for (let i = 0; i < lines.length; i++) {
    const line = lines[i];
    if (IGNORE.test(line)) continue;
    const isColumn = COLUMN.test(line);
    if (!isColumn && !(line.includes('{') && INLINE.test(line))) continue;
    // A multi-line column puts its render/link on the following lines; stop at
    // the line that closes it so the NEXT column's link is not credited to it.
    let end = i;
    if (isColumn && !/}\\s*,?\\s*$/ .test(line)) {
      while (end < i + LOOKAHEAD && end + 1 < lines.length) {
        end++;
        if (/}\\s*,?\\s*$/ .test(lines[end])) break;
      }
    }
    const window = lines.slice(i, end + 1).join('\\n');
    if (!REGISTRY.test(window)) {
      findings.push({
        kind: HAND_LINK.test(window) ? 'HAND URL' : 'DEAD END',
        file: relative(process.cwd(), file).replace(/\\/g, '/'),
        line: i + 1,
        text: line.trim().slice(0, 140),
      });
    }
    i = end; // don't report the same column's render line again
  }
  return findings;
}

const keyOf = (f) => `${f.kind}|${f.file}|${f.text}`;
const all = roots.flatMap(files).flatMap(scanFile);

if (writeBaseline) {
```

```

writeFileSync(writeBaseline, JSON.stringify(all.map(keyOf).sort(), null, 2) + '\n');
console.log(`baseline written: ${all.length} known finding(s) -> ${writeBaseline}`);
process.exit(0);
}

const known = baselineFile && existsSync(baselineFile)
  ? new Set(JSON.parse(readFileSync(baselineFile, 'utf8'))) : new Set();
const fresh = all.filter((f) => !known.has(keyOf(f)));

for (const f of fresh) console.log(`${f.kind} ${f.file}:${f.line} ${f.text}`);
const dead = fresh.filter((f) => f.kind === 'DEAD END').length;
const hand = fresh.length - dead;
const baselineNote = known.size ? ` (${all.length - fresh.length} known finding(s) in baseline)` : '';
console.log(fresh.length
  ? `\n${dead} dead end(s), ${hand} hand-written URL(s)${baselineNote}\n`
  + 'Fix: render identifiers with <RecordLink type id> / linkTo(); mark real exceptions "// nav-ok: <reason>".'
  : `check-record-links: PASS${baselineNote}`);
process.exit(fresh.length ? 1 : 0);

```

9. AI tooling files (.md): skill, agents, commands, rules

The kit folder No-Dead-Ends-Kit is laid out so it can be copied into any project's root:

Path	What it is	Read by
.claude/skills/no-dead-ends/SKILL.md	The skill. Claude loads it automatically when the work involves lists, detail pages or navigation.	Claude Code
.claude/skills/no-dead-ends/references/*.md	rules.md, prompts.md, commands.md, checklist.md, test-scenarios.md	The skill, people
.claude/skills/no-dead-ends/templates/*	The Section 7 code	Copied into the project
.claude/skills/no-dead-ends/scripts/check-record-links.mjs	The Section 8 audit script	Hooks, CI, people
.claude/agents/navigation-auditor.md	Read-only audit subagent (P1 to P3 plus detail-page review)	Claude Code
.claude/agents/navigation-fixer.md	Implementation subagent (P4 to P8)	Claude Code
.claude/commands/nav-audit.md, nav-fix.md	The /nav-audit and /nav-fix slash commands	Claude Code
.claude/hooks/*.mjs, .claude/settings.json	Hooks (Section 10)	Claude Code
AGENTS.md	The rules as agent instructions	Codex, Cursor, Copilot, Gemini and other agents
CLAUDE.md.snippet	The same rules, for pasting into CLAUDE.md	Claude Code
.cursor/rules/no-dead-ends.mdc	Cursor project rule (applies to UI files)	Cursor
.github/hooks/pre-commit, .gitattributes	Git hook and line-ending safety	Git, every developer
README.md	Install and usage guide	People

9.1 Install into a project

1. **Claude Code files.** Copy .claude/ into the project root. If .claude/settings.json already exists, merge its hooks block instead of overwriting the file.
2. **AGENTS.md.** Copy AGENTS.md to the root, or merge its section into the existing file.

3. **CLAUDE.md.** Paste CLAUDE.md.snippet into the project's CLAUDE.md.
4. **Cursor (optional).** Copy .cursor/rules/.
5. **Git hook.** Copy .githooks/ and .gitattributes, then run `git config core.hooksPath .githooks` once per clone.
6. **Domain tuning.** Edit ENTITIES in the audit script to match your entities.
7. **Existing app.** Write the baseline (Section 10.4) and commit .nav-baseline.json.
8. **Machine-wide option.** To use the skill, agents and commands in every project on the machine, copy them into `~/.claude/skills/`, `~/.claude/agents/` and `~/.claude/commands/`.

9.2 SKILL.md

```

---
name: no-dead-ends
description: Navigation and record-page standard for business applications (CRM, ERP, portals,
admin panels, any app with lists and detail pages). Use whenever building or reviewing list
screens, tables, detail/360 pages, related-record tabs, timelines, notifications, search results or
dashboards; when a user reports rows or IDs that cannot be clicked, records that cannot be opened,
a loading skeleton that never finishes, an empty overview tab, or content clipped at the edge; and
when asked to audit navigation, add links to records, or build a detail page. Enforces: every
record shown anywhere opens that record through one permission-aware link component, every panel
resolves (data/empty/error/forbidden), and every detail page follows a standard layout.
---

# No Dead Ends

**Rule:** every record shown anywhere is a way into that record, and every panel finishes loading.

A registration number, invoice number or customer name that looks like an identifier but cannot be
clicked forces users to copy it into search. A skeleton that never resolves hides a failed or
missing request. Both are defects, not polish.

## When building anything that displays records

1. Before the first list screen, create the route registry and RecordLink component (copy
`templates/entityRoutes.js` and `templates/RecordLink.jsx`, adapt names to the project's router,
auth hook and styling).
2. Render every entity identifier through `RecordLink` (or `linkTo` for programmatic navigation).
Never hand-write record URLs in screens.
3. Make list rows clickable and keep the identifier a real anchor (`templates/ClickableRow.jsx`).
4. Load every panel with a timeout and explicit empty/error branches (`templates/usePanel.js`).
5. Build detail pages to the standard layout (`templates/DetailPage.jsx`, rules NAV-13..NAV-17).
6. APIs return related records as `{ type, id, label }` (`templates/api-reference-shape.json`).
7. Add the guards: `scripts/check-record-links.mjs` in CI and `templates/navigation.spec.js` as an
E2E test.

## When auditing or fixing an existing app

Follow `references/prompts.md` steps P1 to P8 in order, stopping after P3 for approval:

- P1 Inventory: entity types, detail routes, by-id endpoints, permissions.
- P2 Dead-end audit: every place an entity is displayed, and whether it is linked. Start with
`node <skill-dir>/scripts/check-record-links.mjs client/src`. It is a heuristic, so also read list
and detail screens yourself.
- P3 Hanging-loader audit.
- P4 onwards: build the foundation, replace dead ends, fix loaders and APIs, standardise detail
pages, add guards, then verify.

Report before/after counts (displayed / linked / dead / hand-written URLs / hanging loaders) with
real command output.

## Existing apps: adopt without blocking work

Existing apps usually start with many findings. Record them once as the baseline:

```

```
```bash
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --write-baseline .nav-
baseline.json
```
```

From then on, the hooks and CI fail only on **new** dead ends. Burn the baseline down screen by screen, and regenerate it after each fix batch so it only ever shrinks.

Hooks (installed with the kit)

- **PostToolUse** (``Edit|Write|MultiEdit``) runs ``.claude/hooks/nav-check-edit.mjs``.
 - It checks each edited UI file.
 - A new dead end is sent back to you (exit code 2). Fix it before moving on.
- **Stop** runs ``.claude/hooks/nav-check-stop.mjs``.
 - When UI files changed, it audits the whole client before you finish.
 - New dead ends block the stop. Fix them, or mark a genuine exception with ``// nav-ok: <reason>``.
- **Git pre-commit** (``.github/hooks/pre-commit``, enabled with ``git config core.hooksPath .github/hooks``) runs the same check on staged files for every developer and tool.

Non-negotiables

- **Permission-aware links.** No access means plain text, never a link that ends on a 403 or 404 page. The server still returns 404 for out-of-scope records.
- **Unknown entity types throw.** An unknown type in ``linkTo`` or ``RecordLink`` must throw an error. A silent ``#`` link is itself a dead end.
- **Missing detail pages are gaps, not fake links.** If an entity has no detail page yet, show plain text and log it in the gap register.
- **List state lives in the URL.** Filters, sort, page and tab go in the URL, so Back restores the list and views can be shared.
- **Nothing is clipped.** At 1366 px with the sidebar open, wide content scrolls inside its own container.
- **Evidence, not claims.** Never claim "fixed" without the audit script and E2E output.

Files

| File | Use |
|---|---|
| --- --- | |
| <code>`references/rules.md`</code> | Rules NAV-01..NAV-20 (paste into specs) |
| <code>`references/prompts.md`</code> | Master prompt and step prompts P1 to P8 |
| <code>`references/commands.md`</code> | Audit commands (rg and PowerShell) |
| <code>`references/checklist.md`</code> | Definition of Done for navigation |
| <code>`references/test-scenarios.md`</code> | Test scenarios T1 to T16 |
| <code>`templates/*`</code> | Reference code for React, Router and Playwright; adapt to the stack |
| <code>`scripts/check-record-links.mjs`</code> | CI guard; exits 1 on dead ends or hand-written URLs |

9.3 AGENTS.md

AGENTS.md

Instructions for AI coding agents working in this repository. Merge this section into an existing AGENTS.md if the project already has one.

Navigation: no dead ends

Every record shown anywhere must open that record, and every panel must finish loading.

- **Link every record.** Render every entity identifier (customer, asset, order, invoice, ticket, employee, document) through the shared ``RecordLink`` component or the ``linkTo(type, id)`` helper. Never hand-write record URLs in screens. The route registry (``entityRoutes``) is the only place record URLs are defined.
- **Rows and identifiers.** List rows are clickable. The identifier is a real anchor, and row actions stop click propagation.

- **Permissions.** Links respect permissions: a user without access sees plain text, never a link to 403 or 404. The server still returns 404 for out-of-scope records.
- **360 pages.** Tabs link to child records, and children link back to their parents. Timeline, notification, search and audit items deep-link to their records.
- **API references.** APIs return related records as `{ type, id, label }`.
- **Loaders.** Every panel resolves to data, empty, error-with-retry or forbidden within 12 s. An endless skeleton is a bug.
- **Detail pages.** Standard header (name, business ID, status, owner, key facts, actions); an Overview that answers who / what is open / what next; tabs ordered by use with counts and muted zero tabs; breadcrumbs; list state in the URL; nothing clipped at 1366 px with the sidebar open.
- **Missing detail pages.** An entity with no detail page yet is shown as plain text and logged in the gap register.
- **Before finishing UI work,** run `node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --baseline .nav-baseline.json` (must PASS) and the navigation E2E test. Mark deliberate exceptions with `// nav-ok: <reason>`.

Full rules NAV-01 to NAV-20 are in `.claude/skills/no-dead-ends/references/rules.md`.

10. Hooks: automatic enforcement

Rules in a document get forgotten; hooks enforce them every time. The kit installs three hooks, all of which run the Section 8 audit script. Each one skips quietly when the script is missing or when no UI file changed. They need Node.js 18 or later.

| Hook | When it runs | What it does |
|---|--|---|
| PostToolUse (Claude Code) nav-check-edit.mjs | After every Edit, Write or MultiEdit of a .jsx, .tsx, .vue or .svelte file | Checks that file. A new dead end is fed straight back to the agent (exit code 2), which fixes it while the file is still open. |
| Stop (Claude Code) nav-check-stop.mjs | When the agent is about to report that it is finished | If UI files changed, it audits the whole client. New dead ends block the stop, so the agent cannot claim "done". It never loops: stop_hook_active is respected. |
| pre-commit (git) .github/hooks/pre-commit | On every git commit, by any person or tool | Checks the staged UI files and blocks the commit when new dead ends are found. |

10.1 .claude/settings.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node \"$CLAUDE_PROJECT_DIR/.claude/hooks/nav-check-edit.mjs\"",
            "timeout": 30
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "node \"$CLAUDE_PROJECT_DIR/.claude/hooks/nav-check-stop.mjs\"",
            "timeout": 60
          }
        ]
      }
    ]
  }
}
```



```

    ]
  }
}

```

10.2 PostToolUse hook: .claude/hooks/nav-check-edit.mjs

```

// Claude Code PostToolUse hook (Edit | Write | MultiEdit).
// After the agent edits a UI file, check just that file for dead ends and feed
// any NEW finding back to the agent (exit 2 = stderr is shown to Claude), so it
// fixes the link while the file is still in front of it.
import { spawnSync } from 'node:child_process';
import { existsSync, readFileSync } from 'node:fs';
import { join } from 'node:path';

const project = process.env.CLAUDE_PROJECT_DIR || process.cwd();
const checker = join(project, '.claude/skills/no-dead-ends/scripts/check-record-links.mjs');
const baseline = join(project, '.nav-baseline.json');

let input = {};
try { input = JSON.parse(readFileSync(0, 'utf8') || '{}'); } catch { process.exit(0); }
const file = input.tool_input?.file_path;
if (!file || !/\.(\.jsx|tsx|vue|svelte)$/.test(file) || !existsSync(checker)) process.exit(0);

const args = [checker, file];
if (existsSync(baseline)) args.push('--baseline', baseline);
const run = spawnSync(process.execPath, args, { cwd: project, encoding: 'utf8' });
if (run.status === 1) {
  process.stderr.write(`No-dead-ends check failed for ${file} (rules NAV-01/NAV-02):\n${run.stdout}`);
  process.exit(2);
}
process.exit(0);

```

10.3 Stop hook: .claude/hooks/nav-check-stop.mjs

```

// Claude Code Stop hook.
// Before the agent reports a task finished, audit the whole client for NEW dead
// ends. If any exist, block the stop (exit 2) and hand the list back so the
// agent fixes them instead of claiming done.
import { spawnSync } from 'node:child_process';
import { existsSync, readFileSync } from 'node:fs';
import { join } from 'node:path';

const project = process.env.CLAUDE_PROJECT_DIR || process.cwd();
const checker = join(project, '.claude/skills/no-dead-ends/scripts/check-record-links.mjs');
const baseline = join(project, '.nav-baseline.json');

let input = {};
try { input = JSON.parse(readFileSync(0, 'utf8') || '{}'); } catch { /* run anyway */ }
// Already continuing because of this hook: let it stop rather than loop forever.
if (input.stop_hook_active || !existsSync(checker)) process.exit(0);

// Only spend the time when UI files changed (skip for docs-only or backend-only work).
const git = spawnSync('git', ['status', '--porcelain'], { cwd: project, encoding: 'utf8' });
if (git.status === 0 && !/\.(\.jsx|tsx|vue|svelte)$*/m.test(git.stdout)) process.exit(0);

// Client source folder: NAV_CLIENT_SRC, or the first common layout that exists.
const src = process.env.NAV_CLIENT_SRC
  || ['client/src', 'src', 'web/src', 'apps/web/src', 'frontend/src'].find((d) =>
existsSync(join(project, d)));
if (!src) process.exit(0);

```

```
const args = [checker, src];
if (existsSync(baseline)) args.push('--baseline', baseline);
const run = spawnSync(process.execPath, args, { cwd: project, encoding: 'utf8' });
if (run.status === 1) {
  process.stderr.write('Not done yet: new navigation dead ends were introduced (no-dead-ends skill).\n'
    + `${run.stdout}\nFix them, or mark genuine exceptions with "// nav-ok: <reason>".\n`);
  process.exit(2);
}
process.exit(0);
```

10.4 Git pre-commit hook and the baseline

```
#!/bin/sh
# Git pre-commit hook: block commits that add navigation dead ends (no-dead-ends kit).
# Enable once per clone: git config core.hooksPath .githooks
# Works for every developer and every AI tool, not only Claude Code.

CHECKER=".claude/skills/no-dead-ends/scripts/check-record-links.mjs"
[ -f "$CHECKER" ] || exit 0

# Staged UI files only (added, copied, modified, renamed).
FILES=$(git diff --cached --name-only --diff-filter=ACMR | grep -E '\.(jsx|tsx|vue|svelte)$')
[ -n "$FILES" ] || exit 0

BASELINE=""
[ -f ".nav-baseline.json" ] && BASELINE="--baseline .nav-baseline.json"

# shellcheck disable=SC2086
if ! node "$CHECKER" $FILES $BASELINE; then
  echo ""
  echo "Commit blocked: new navigation dead ends (see above).\"
  echo "Render identifiers with <RecordLink type id> / linkTo(), or mark a real exception // nav-
ok: <reason>."
  echo "Bypass only with a reason: git commit --no-verify"
  exit 1
fi
exit 0
```

Baseline for existing apps. An existing app usually starts with many findings. Failing on all of them would block every change, so record them once:

```
node .claude/skills/no-dead-ends/scripts/check-record-links.mjs client/src --write-baseline .nav-
baseline.json
```

Commit `.nav-baseline.json`. From then on, the hooks and CI fail only on **new** dead ends. Fix the old ones screen by screen, and regenerate the baseline after each batch so it only ever shrinks.

Verified

The hooks passed 8 tests in a sandbox project:

- The edit hook blocks a new dead end, and passes linked files and non-UI files.
- The stop hook respects `stop_hook_active`, blocks a new dead end, passes when every finding is in the baseline, and blocks again when one new dead end appears.

The git hook allowed a commit of a linked file and blocked a commit that added a dead end.

11. Navigation map template

Fill this in once per project (step P1 produces it) and keep it with the docs. Every row must have a route and an endpoint, or be logged as a gap.

| Entity type | Label / business ID | Detail route | By-id endpoint | Permission | Shown on | Status |
|-------------|---------------------|----------------|------------------------|----------------|--|------------|
| customer | name / number | /customers/:id | GET /api/customers/:id | customers.view | Customers list, search, orders, timeline | Linked |
| asset | registration | /assets/:id | GET /api/assets/:id | assets.view | Customer 360 › Assets, renewals, quotes | DEAD → fix |
| {{entity}} | | | | | | |

12. Definition of Done: navigation

- ☐ Navigation map complete: every entity type has a detail route, a by-id endpoint and a permission (or is logged as a gap).
- ☐ Route registry and RecordLink exist; the audit script reports 0 hand-written URLs.
- ☐ Every displayed entity is linked or deliberately plain text; the audit script passes (exceptions carry a nav-ok: reason).
- ☐ Rows clickable; the identifier is a real anchor; row actions do not trigger navigation.
- ☐ Links hidden for users without permission; the destination still returns 404 for out-of-scope records.
- ☐ 360 tabs link to children; children link back to parents; tab counts equal rows.
- ☐ Timeline, notifications, search results and audit rows deep-link to their records.
- ☐ Every panel resolves within the timeout to data, empty, error-with-retry or forbidden; no aria-busy after 15 s.
- ☐ Detail pages have the standard header, Overview, ordered tabs with muted zero tabs, and breadcrumbs.
- ☐ Nothing clipped at 1366 px with the sidebar open, or at 390 px on mobile.
- ☐ Back restores list filters, sort, page and scroll; detail URLs work on refresh and when shared.
- ☐ E2E navigation test green for every role; output attached.

13. Test scenarios

| # | Scenario | Expected |
|----|---|--|
| T1 | Click the identifier in the first row of every list, as each role | Opens that record's detail page |
| T2 | Click elsewhere in the row | The same record opens (one navigation, not two) |
| T3 | Middle-click / Ctrl+click the identifier; copy link address | Opens in a new tab; the copied URL opens the record |
| T4 | Tab to the identifier and press Enter (keyboard only) | Record opens; focus visible |
| T5 | User without permission for the child entity views the parent 360 | Child identifiers are plain text; no link to a 403/404 |
| T6 | Paste another team's record URL directly | Server returns 404; no data shown |
| T7 | Open each 360 tab; click its first row; press Back | Child opens; Back returns to the same tab and scroll |
| T8 | Child detail page header | Links back to its parent record(s) |
| T9 | Timeline item, notification, search result, audit row | Each opens the record it refers to |

| # | Scenario | Expected |
|-----|---|--|
| T10 | Timeline API returns 500 / times out / returns [] | Error with retry and request ID / timeout message / "No activity yet"; never an endless skeleton |
| T11 | Filter and sort a list, go to page 3, open a record, press Back | Same filters, sort, page and scroll |
| T12 | Refresh a detail page; open a bookmarked detail URL in a new session | Loads (after sign-in) on the same record |
| T13 | Detail page at 1366 px with the sidebar open; very long names and many badges | Nothing clipped; overflow scrolls in its container |
| T14 | Tab with zero records | Muted, reachable, empty state with create action |
| T15 | Open a record that was merged into another | Redirects to the surviving record |
| T16 | KPI tile on a dashboard | Opens the filtered list; count equals rows; rows open records |

The automated version of T1, T7 and T10 is the Playwright test in Section 7.7.